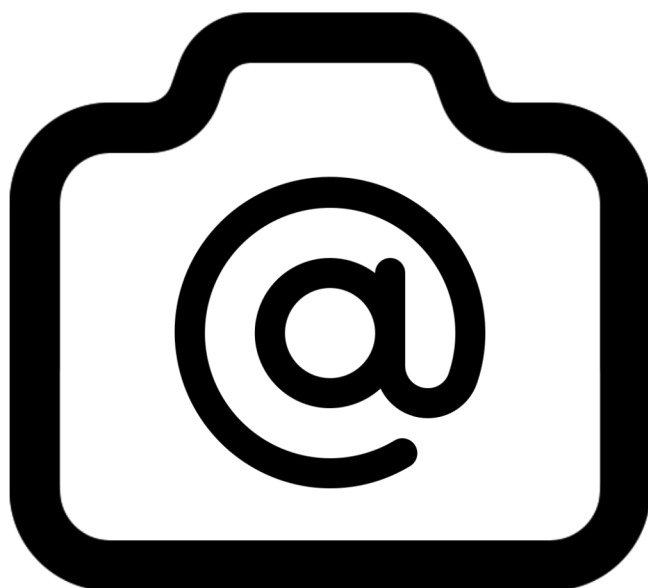


YouToText



Indice

1. Introducción.....3

2. Objetivos.....4

 Objetivo General.....4

 Objetivos Específicos.....4

3. Análisis y estado actual del proyecto.....5

 3.1 Requisitos Técnicos Identificados.....5

 3.2 Implementación Actual.....5

1. Introducción

YouToText se desarrolla como proyecto académico de fin de ciclo con el objetivo de explorar el procesamiento de imágenes en tiempo real sobre dispositivos Android. La aplicación captura el flujo de la cámara, analiza cada fotograma y lo transforma en una representación visual basada en caracteres ASCII estándar. Para ello, se emplea la biblioteca oficial **CameraX**, que gestiona la exposición de frames y el ciclo de vida de la cámara, junto con **Kotlin** y mecanismos de ejecución en segundo plano para evitar bloqueos en la interfaz.

El proyecto se centra en tres ejes técnicos: la implementación de un analizador de frames (`ImageAnalysis.Analyzer`) que extraiga datos de luminancia, el mapeo de dichos valores a un conjunto de caracteres configurable y la exportación del resultado a portapapeles, imagen PNG o vídeo. Se prioriza la ejecución local, el consumo controlado de memoria y el mantenimiento de una tasa de refresco estable, así como la organización progresiva del código para facilitar su mantenimiento y escalabilidad.

2. Objetivos

Objetivo General

Desarrollar una aplicación Android que transforme el flujo de vídeo de la cámara en una secuencia de texto ASCII en tiempo real, gestionando correctamente los permisos, el ciclo de vida de CameraX y el procesamiento de imágenes en hilos secundarios.

Objetivos Específicos

1. Configurar y vincular los casos de uso de CameraX (Preview, ImageCapture, VideoCapture) al ciclo de vida de la actividad, garantizando la liberación adecuada de recursos.
2. Implementar un ImageAnalysis.Analyzer que acceda al plano Y (luminancia) de cada ImageProxy, calcule su valor promedio por bloque y lo traduzca a caracteres ASCII estándar (@%#*+=-:.).
3. Ejecutar el pipeline de conversión en corrutinas para evitar bloqueos en el hilo principal y mantener un rendimiento objetivo de **>20 FPS**.
4. Integrar rutinas de exportación: copia de la cadena ASCII al portapapeles del sistema, generación de imágenes PNG mediante Bitmap y grabación de vídeo con VideoCapture.
5. Estructurar el proyecto de forma modular, separando la lógica de cámara, el procesamiento de frames y la interfaz, para facilitar pruebas y futuras mejoras.

3. Análisis y estado actual del proyecto

3.1 Requisitos Técnicos Identificados

- **API objetivo:** Android 8.0 (API 26) para garantizar compatibilidad con CameraX y MediaStore.
- **Permisos:** CAMERA, RECORD_AUDIO (opcional para vídeo) y WRITE_EXTERNAL_STORAGE, tal como se declara en AndroidManifest.xml.
- **Hardware:** Requiere cámara trasera o frontal (android.hardware.camera.any) y soporte para procesamiento de imágenes en tiempo real.
- Rutas de almacenamiento:
 - Fotos: Pictures/YouToText-Images
 - Vídeos: Movies/YouToText-Videos
 - Se utiliza MediaStore con ContentValues para evitar accesos directos al sistema de archivos en API ≥ 29 .

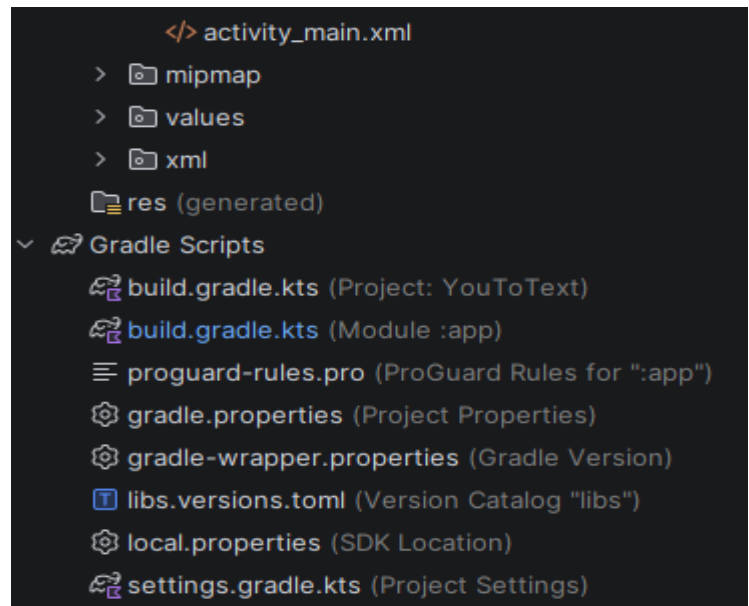
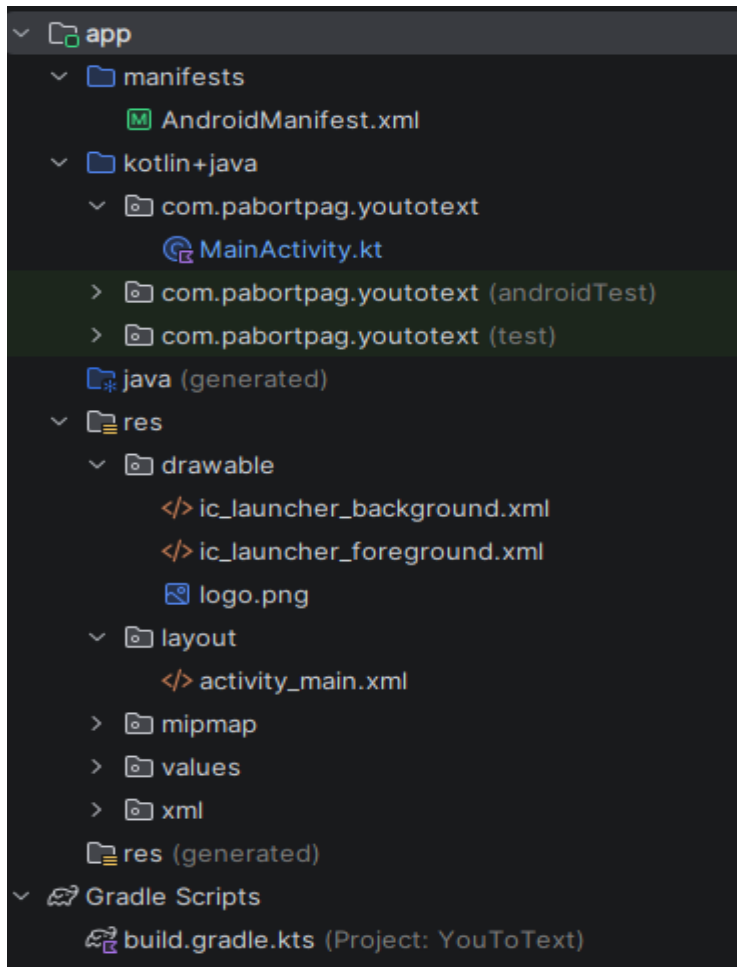
3.2 Implementación Actual

El código base ya incluye:

- ☑ Solicitud de permisos mediante `ActivityResultContracts.RequestMultiplePermissions()`.
- ☑ Captura de fotos y vídeos con timestamp, guardados en MediaStore y notificación al usuario vía Toast.
- ☑ Guardado de fotos y vídeos en MediaStore con nombres timestamp y rutas relativas (Pictures/YouToText-Images, Movies/YouToText-Videos).
- ☑ Uso de un `ExecutorService` de un solo hilo para tareas en segundo plano.
- ⚠ **Pendiente:** Habilitar y adaptar `ImageAnalysis` (actualmente comentado en `startCamera()`).
- ⚠ **Pendiente:** Migrar la lógica de cámara fuera de `MainActivity` para evitar acoplamiento.

3.3 Arquitectura propuesta (estructura inicial)

Para cumplir con los objetivos de rendimiento y mantenimiento, se plantea la siguiente organización de paquetes:



Esta estructura permite aislar el procesamiento de frames, facilita la inyección de dependencias en fases avanzadas y garantiza que el `analyze()` de `ImageAnalysis` no interfiera con la UI.

4. Evaluación de los resultados

5. Conclusiones

6. Recursos

- Andorid Studio
- CameraX
- GitHub

7. Bibliografía

[Pagina Oficial CameraX](#)

8. Anexos